tensor4all-rs: tree methods and adaptive patching

Progress from 22 September to 27 October 2026

Lingrui Cheng

tensor4all developer meeting, 27 October 2026

With Hiroshi Shinaoka and the tensor4all developers

github.com/tensor4all/tensor4all-rs

Since September 22

tensor4all-rs

New capabilities and reliability work

Rust implementation in tensor4all-rs

- Tree adaptive patching: generalization of the TT interpolation method.
- TreeRSI: a new tree generalization with reference-code checks.
- TreeTCI and TreeACI: numerical fixes, convergence checks and bounded caches.

Rust library: github.com/tensor4all/tensor4all-rs

ACI: M. K. Ritter, arXiv:2604.00037

SRC: Camaño et al., arXiv:2504.06475

Adaptive patching on trees

tensor4all-rs

Tree interpolation

- One interpolation driver supports chains and branched trees.
- Split difficult regions while preserving the original tree topology.
- Reuse parent samples and pivots in the child patches.

Accuracy and split controls

- Measure patch errors against a fixed global L2 reference.
- Patch-size controls prevent excessive subdivision.
- Reuse large cached values as child candidates to reduce missed features.

Sampled checks can miss localized residuals. Exhaustive checks support certificates.

Grosso et al., Adaptive Patching for Tensor Train Computations, arXiv:2602.22372

Patching integration and execution

tensor4all-rs

Library integration

- Chains and branching topologies use the same tree implementation.
- The old TT-only patching crate has been retired.
- A shared engine interface keeps the patch driver independent of TreeTCI.

Parallel interpolation

- Shinaoka added local and MPI execution after the tree driver merged.
- Independent patches can run across workers or MPI ranks.
- Small consistency tests pass. Performance scaling still needs measurement.

Tree driver: L. Cheng (#900). Local/MPI integration: H. Shinaoka (#1005).

Scaling and reproducibility across all supported tree layouts still need validation.

Current tree driver and execution contract

TreeRSI

tensor4all-rs

Tree generalization

- RSI for simultaneous Hadamard products.
- Extends the original TT algorithm directly to branched trees.
- Uses sketches to avoid forming the full high-rank product.

Validation and limits

- Checked against the authors' TT code and against TreeACI on branched cases.
- New timings: 2.9–4.8× faster than current TreeACI in four synthetic fixtures.
- These fixtures allow full reconstruction. Tighter-rank compression still needs study.

Experimental: accuracy at a fixed rank and variation across seeds remain limitations.

Meng, Khoo, Li and Stoudenmire, arXiv:2602.17974

Reliability of the tree methods

tensor4all-rs

Search and convergence

TreeTCI keeps search improvements and processes new pivots before stopping. TreeACI rechecks previously detected failures.

Tree structure

Site-free junctions can grow their ranks. Basic TreeTN operations now handle nodes with no physical sites.

Numerical reliability

Preserve small nonzero contributions, reject NaN/Inf and make tolerance handling consistent across scales.

Fixes by L. Cheng, with later site-free operation support completed by H. Shinaoka.

Evaluation and memory

tensor4all-rs

Repeated network readout

- Reuse cached subtree contractions when reading the current approximation.
- Fully cached queries avoid rebuilding work already available.
- Bound buffers and cache growth to reduce memory pressure.

Expensive target functions

- Optionally reuse target values across interpolation stages.
- Default off. Useful when each function evaluation is costly.
- Synthetic benchmark: 12.8× faster overall, versus 4.2× with memo off.

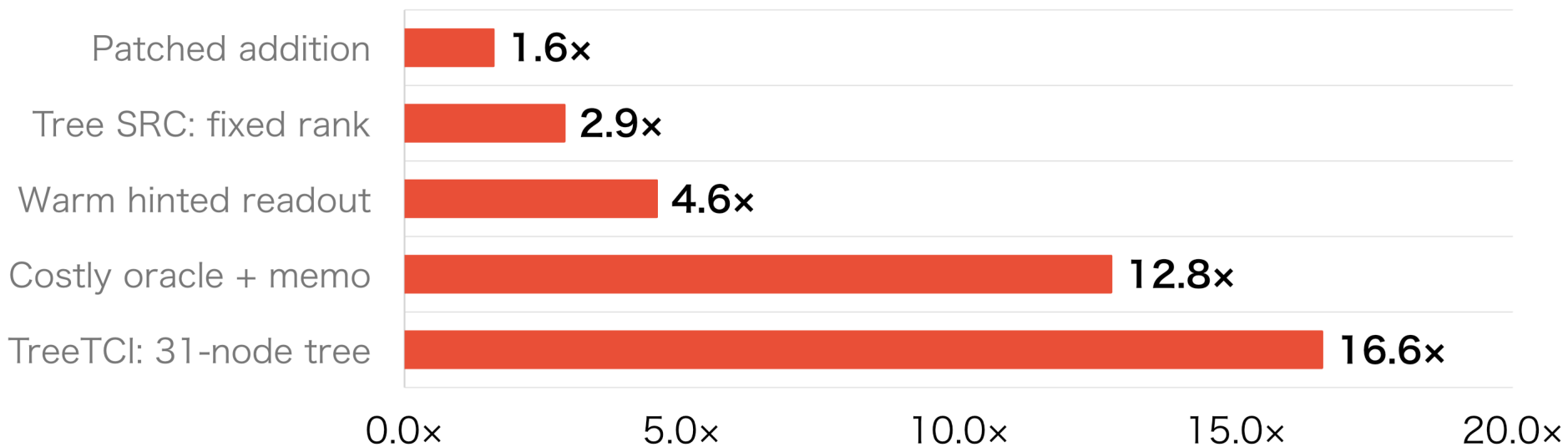
The two caches save network readout and target evaluation, respectively.

L. Cheng: cached readout (#798, #865), target memo (#852) and memory bounds (#846).

Benchmark: improvements

tensor4all-rs

September baseline 4fee1a32 versus October main 9222cbee



Speedup = old median time / new median time

Ryzen 9 6900HX, one CPU thread, release. Five alternating rounds; independent error checks.

Version differences include the whole team and backend updates.

Benchmark: costs and limits

tensor4all-rs

TreeACI: checking costs

High-rank products take 1.5–2.1× longer.
Guard now runs even without injection headroom.

Cache defaults

Comb χ=64: 1.3–1.5× slower.
Same cache/chunk settings: about 1.0×.

Patching counterexample

3D spectral function, η=0.03.
Old mono TCI versus new patched TCI.

27.9 s to 111.0 s

Sampled relative L2:
3.1e-5 to 3.8e-4

5.8× fewer oracle points,
5.0× more stored parameters.

Errors differ in the patching comparison. Report time and accuracy together.

Patching: 24 binary sites, cap 64, 56 patches; 4,000 independent check points.

Remaining work

tensor4all-rs

- Patched MPO-MPO: complete the adaptive contraction path, with early stopping, selective refinement and sibling merging.
- Accuracy: address localized residuals that escape sampled checks, and complete algebra error accounting.
- TreeACI: resolve convergence issues and profile the cost of Guard checks.
- Parallel execution: measure scaling of the existing local/MPI interpolation and extend the contraction side.

Tree interpolation is in main. Full adaptive patched algebra remains the largest gap.